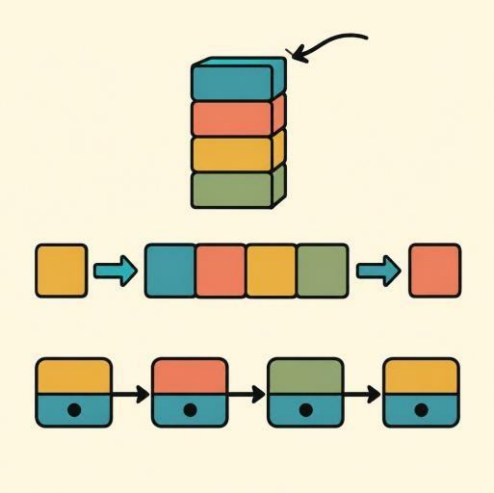
Estructuras de Datos Lineales: Listas, Pilas y Colas

Conceptos Clave

Pila (Stack) — estructura LIFO (Last In, First Out): el último elemento en entrar es el primero en salir.
Operaciones: `push` (apilar) y `pop` (desapilar).

Cola (Queue) — estructura FIFO (First In, First Out): el primer elemento en entrar es el primero en salir.
Operaciones: `enqueue` (encolar) y `dequeue` (desencolar).

Lista Enlazada — secuencia de nodos donde cada nodo guarda un dato y una referencia al siguiente.
Operaciones: `add` (insertar) y `remove` (eliminar).



Imagina tres escenas cotidianas: una pila de platos donde solo puedes tomar el de arriba, una fila del banco donde el primero en llegar es el primero en ser atendido, y una lista de reproducción donde puedes saltar a cualquier canción. Cada escena es una estructura de datos lineal.

Una **pila** solo permite trabajar por un extremo, llamado tope. Cuando apilas Las tres estructuras lineales básicas con `push`, el nuevo elemento queda encima de todos; cuando desapilas con `pop`, sale siempre el último que entró.

Ese orden se conoce como **LIFO**.

Una **cola** trabaja por los dos extremos: los elementos entran por atrás con `enqueue` y salen por el frente con `dequeue`. El primero en llegar es el primero en salir: **FIFO**.

Una **lista enlazada** guarda sus elementos en nodos conectados por referencias. Puedes insertar con `add` o eliminar con `remove` en cualquier posición recorriendo los nodos desde el inicio, lo que la hace ideal cuando el orden de los datos cambia con frecuencia.

Caso 1: Una Pila para revertir una cadena de texto

Para revertir una cadena, recorre sus caracteres de izquierda a derecha y haz `push` de cada uno en la pila. Luego haz `pop` repetidamente: como el último carácter en entrar es el primero en salir, la cadena sale invertida.

Completa los métodos `push` y `pop` del esqueleto:

```
public class PilaCaracteres
{      private char[] pila = new
char[20];      private int tope = -1;
         public void push(char
c) {
        // COMPLETA: coloca c en la posición tope + 1
        // y actualiza el valor de tope
    }      public char
pop() {
        // COMPLETA: devuelve el carácter en la posición tope,
        // y actualiza tope para "sacarlo" de la pila
    }     public String revertir(String texto) { String resultado
= "";         for (int i = 0; i < texto.length(); i++)
{             push(texto.charAt(i));
        }         while
(tope >= 0)
{              resultado +=
pop();
    }         return
resultado;
    }
}
```

Escribe tu implementación de los métodos `push` y `pop` :

Explica paso a paso qué hace la pila al ejecutar `revertir("HOLA")` : indica qué carácter entra con cada `push` y en qué orden salen con cada `pop` .

Caso 2: Una Cola para gestionar una fila de impresión

Una impresora atiende los documentos en el orden en que llegaron: el primero en la fila es el primero en imprimirse. Cada documento se incorpora con `enqueue` al final de la cola y sale con `dequeue` por el frente. Completa los métodos `enqueue` y `dequeue` :

```
public class ColaImpresion {
    private String[] documentos = new String[10];

    private int frente = 0;
    private int fin = -1; // fin indica el último documento encolado

    public void encolar(String doc) {
        // COMPLETA: agrega doc al final de la cola
        // y actualiza fin

    }
    public String desencolar() {
        // COMPLETA: devuelve el documento en la posición frente
        // y actualiza frente para pasarlo al siguiente
    }      public void
imprimirTodo() {         while
(frente <= fin) {
            System.out.println("Imprimiendo: " + dequeue());
        }
    }
}
```

Escribe tu implementación de los métodos `enqueue` y `dequeue` :

Llegan los documentos en este orden: `"Tarea"` , `"Examen"` , `"Boleta"` . Explica paso a paso cómo se ejecutan las operaciones: qué hace cada `enqueue` al llegar y qué imprime cada `dequeue` , indicando en todo momento quién está al frente de la cola.

Caso 3: Una Lista para un historial de navegación

El historial de un navegador guarda las páginas visitadas en orden. Con `add` se inserta cada nueva página al final de la lista; con `remove` se puede borrar una página concreta recorriendo los nodos desde el inicio. Completa los métodos `add` y `remove` :

```
public class HistorialNavegacion {     private
String[] paginas = new String[15];
     private int tamanio = 0; // cuántas páginas hay
guardadas
     public void add(String url)
{
        // COMPLETA: guarda url en la posición indicada por tamaño
        // y aumenta tamaño en 1
    }      public void remove(String
url) {
        // COMPLETA: busca url recorriendo el arreglo desde el índice 0;
        // si la encuentras, desplaza una posición a la izquierda
        // todas las páginas posteriores y reduce tamaño en 1
    }      public void mostrarHistorial()
{         for (int i = 0; i < tamanio;
i++) {
             System.out.println((i + 1) + ". " + paginas[i]);
        }
    }
}
```

Escribe tu implementación de los métodos `add` y `remove` :

El historial empieza vacío y se ejecutan: `add("escuela.edu")` , `add("biblioteca.org")` , `add("noticias.com")` , `remove("biblioteca.org")` . Explica paso a paso cómo queda el historial después de cada operación y por qué la lista (a diferencia de la pila o la cola) permite eliminar un

elemento que está en medio.

Para recordar

Completa el cuadro comparativo con lo que repasaste en esta actividad:

Estructura	Principio	Operaciones	Caso práctico
Pila			
Cola			
Lista Enlazada			

Nombre: Hernández Exiga Jesús Manuel **Fecha:** 13/11/26 **Clase:** GTID244

Algoritmos de Ordenamiento por Etapas

Conceptos clave

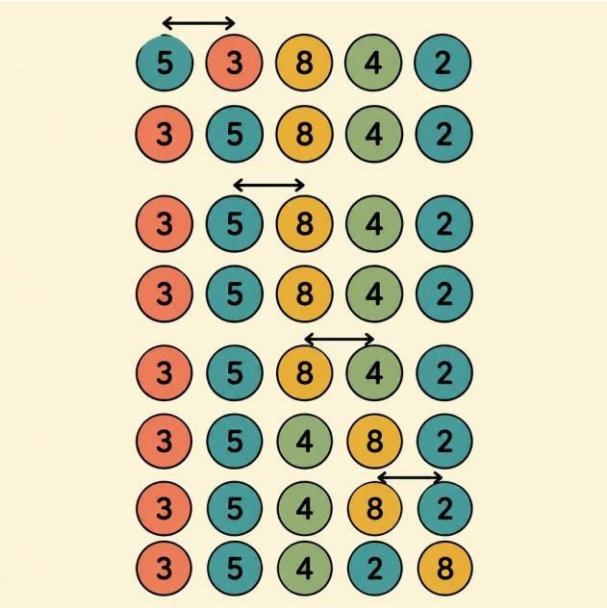
Bubble Sort (ordenamiento burbuja) — recorre el arreglo comparando elementos vecinos y los intercambia si están en el orden incorrecto. En cada pasada, el valor más grande "burbujea" hasta su posición final. Su principio es simple, pero hace muchas comparaciones.

QuickSort (ordenamiento rápido) — elige un elemento llamado **pivote**, divide el arreglo en dos partes (menores y mayores que el pivote) y repite el proceso en cada parte. Es más eficiente porque hace menos comparaciones.

Pasada (iteración) — una vuelta completa del algoritmo sobre el arreglo (o sobre una partición, en QuickSort).

Intercambio (swap) — mover un valor de una posición a otra usando una variable temporal.

Imagina que te entregan una pila de 8 exámenes en orden aleatorio y debes devolverlos ordenados por calificación. Podrías comparar exámenes vecinos dos a dos, intercambiarlos cuando estén mal colocados y repetir el recorrido hasta que nada se mueva: eso es Bubble Sort. O podrías elegir un examen como referencia, separar la pila en "menores" y "mayores" que él, y repetir la separación en cada grupo: eso es QuickSort.



Bubble Sort compara e intercambia vecinos

Los dos llegan al mismo arreglo ordenado, pero por caminos distintos. Bubble Sort siempre compara vecinos, mientras que QuickSort salta por el arreglo dividiéndolo. En esta actividad verás cómo se comporta cada uno sobre el mismo arreglo y luego los implementarás en Java.

El arreglo de trabajo

Usa este mismo arreglo en todas las etapas de la actividad:

[42,7,19,3,25,8,15,30]

Índices: posición 0 = 42, posición 1 = 7, posición 2 = 19, posición 3 = 3, posición 4 = 25, posición 5 = 8, posición 6 = 15, posición 7 = 30.

Parte 1 — Bubble Sort: cómo funciona

Bubble Sort recorre el arreglo desde el índice 0 comparando cada par de vecinos. Si el de la izquierda es mayor que el de la derecha, los intercambia. Al terminar la primera pasada, el valor más grande (42) queda en la última posición. En cada pasada siguiente, el siguiente valor más grande queda en su lugar, así que la zona ordenada crece desde el final y cada pasada necesita una comparación menos.

Etapa 1 — Traza visual del Bubble Sort

Completa la tabla con el estado completo del arreglo después de cada pasada. En la pasada 1 se muestra el punto de partida; escribe el arreglo tal como queda tras terminar cada recorrido. Indica también cuántos intercambios ocurrieron en cada pasada.

Pasada	Estado del arreglo al terminar la pasada	Intercambios realizados
Inicio	[42, 7, 19, 3, 25, 8, 15, 30]	0
1	[7, 19, 3, 25, 8, 15, 30, 45]	7

2	[7, 3, 19, 25, 8, 15, 30, 45]	3
3	[3, 7, 19, 25, 8, 15, 30, 45]	1
4	[3, 7, 19, 8, 15, 25, 30, 45]	2
5	[3, 7, 8, 15, 19, 25, 30, 45]	2
6		
7		

1. ¿En qué pasada el arreglo quedó completamente ordenado, aunque el algoritmo aún no lo sabía? ¿Cómo lo notaste? ***En la 5ta iteración, ya estaban ordenados de menor a mayor.***

Parte 2 — QuickSort: cómo funciona

QuickSort no compara vecinos: elige un **pivote** (por ejemplo, el último elemento de la sección que está ordenando) y reorganiza esa sección para que los valores menores queden a la izquierda del pivote y los mayores a la derecha. Ese paso se llama **partición**. Después repite el mismo proceso, por separado, en la parte izquierda y en la derecha, hasta que cada sección tenga uno o cero elementos.

Etapa 1 — Traza visual de la primera partición

Toma el arreglo completo [42, 7, 19, 3, 25, 8, 15, 30] y realiza la primera partición usando 30 (el último elemento) como pivote. Escribe qué valores quedaron a la izquierda del pivote, cuáles a la derecha, y en qué posición final quedó el pivote.

2. Valores menores que el pivote (izquierda): (Valores de izquierda) [7, 19, 3, 25, 8, 15]

3. Valores mayores que el pivote (derecha): (Valores de derecha) [42]

4. Posición final del pivote 30 (índice): 6

5. Escribe el arreglo completo después de esta primera partición (el pivote ya en su lugar): [7, 19, 3, 25, 8, 15, 30, 45]

6. ¿Qué se debe hacer con la sección izquierda [7, 19, 3, 25, 8, 15] después de esta partición?Explica el siguiente paso del algoritmo. ***Se marca un nuevo pivote para que empiece a seleccionar los que son menores al pivote seleccionado y se ordene de nuevo en su nueva posición.***

Parte 3 — Etapa 2: Implementación en Java

Completa el método `ordenar(int[] arr)` con el algoritmo Bubble Sort. Después realiza la traza manual del código para validar los cambios en el arreglo.

7. Completa el esqueleto del método Bubble Sort. Escribe el código completo en el espacio siguiente:

```
public static void ordenar(int[] arr) {
    // TODO: recorre el arreglo con dos ciclos anidados
    // TODO: compara arr[j] con arr[j+1]
    // TODO: si arr[j] > arr[j+1], intercámbialos con una variable temporal
}
```

8. Traza manual de tu código: ejecuta mentalmente el método `ordenar` con el arreglo de trabajo y anota los `i` valores de , `j` y el estado del arreglo en los momentos indicados de la primera pasada.

i	j	Estado del arreglo después de esta comparación
0	0	
0	1	
0	2	
0	3	
0	4	
0	5	
0	6	

9. ¿Cómo sabes, observando tu traza, que el valor 42 ya no se moverá en las pasadas siguientes?Explica qué garantiza el algoritmo al final de cada pasada.

10. Ahora implementa en Java el método de partición de QuickSort. Completa el esqueletousando el último elemento como pivote:

```
public static int particion(int[] arr, int inicio, int fin)
{      int pivote = arr[fin];     int i = inicio - 1;     //
TODO: recorre de inicio hasta fin - 1
    // TODO: si arr[j] <= pivote, incrementa i e intercambia arr[i] con arr[j]
    // TODO: al final, coloca el pivote en su posición correcta y devuelve ese índice
}
```

11. Compara tus dos trazas: ¿cuántas comparaciones hizo Bubble Sort en total para ordenar los 8números, y cuántas hizo QuickSort aproximadamente al dividir el arreglo? ¿Cuál crees que escala mejor cuando el arreglo crece a 1,000 elementos? Justifica tu respuesta.

Nombre: Hernández Exiga Jesús Manuel **Fecha:** 13/11/26

Búsqueda Lineal y Búsqueda Binaria

Conceptos clave

Búsqueda lineal — recorre el arreglo desde el primer elemento hasta el último, comparando uno por uno. Funciona en cualquier arreglo, ordenado o no.

Búsqueda binaria — compara el elemento buscado con el valor del medio y descarta la mitad del arreglo en cada paso. **Solo funciona si el arreglo está ordenado previamente.**

Complejidad temporal (Big O) — mide cuánto crece el trabajo de un algoritmo cuando crece el tamaño de la entrada. O(n) significa trabajo proporcional al tamaño; O(log n) significa que el trabajo crece mucho más lento, porque cada paso reduce el problema a la mitad.

Imagina que buscas la palabra zorro en un diccionario. Nadie empieza en la primera página y avanza palabra por palabra: abres por la mitad, ves que zorro está al final, y saltas a la mitad de esa mitad. Eso es búsqueda binaria.

La búsqueda lineal es lo contrario: revisar cada página desde el inicio hasta encontrar la palabra. Siempre funciona, pero puede tardar mucho. La búsqueda binaria es mucho más rápida, pero tiene una condición obligatoria: **los datos deben estar ordenados antes de buscar**. Si el diccionario estuviera desordenado, abrirlo por la mitad no te diría nada sobre en qué mitad está zorro.

En código, la búsqueda lineal usa un ciclo simple que compara cada elemento con el valor buscado. La búsqueda binaria mantiene tres índices: **inicio** (primera posición del rango), **fin** (última posición) y **medio** (la mitad del rango). En cada iteración compara el valor del medio con el buscado y ajusta inicio o fin, descartando la mitad que ya no puede contener el valor.

Caso práctico

Un sistema debe verificar si el ID de usuario **68** existe en el siguiente arreglo, que ya está ordenado de menor a mayor:

´ındice	0	1	2	3	4	5	6	7
valor	7	13	21	34	42	55	68	79

Etapa 1: Simulación manual de la búsqueda binaria

Direcciones: Completa la tabla con los valores de los índices **inicio**, **fin** y **medio** en cada iteración, el valor encontrado en la posición medio, y la decisión tomada (¿seguir por la mitad izquierda, por la derecha, o terminar?). Recuerda: medio = (inicio + fin) / 2, con división entera.

Iteración	inicio	fin	medio	valor en medio	decisión
1	7	79	42	42	Mitad derecha
2	42	79	68	68	terminar
3					

Pregunta de control: ¿Cuántas comparaciones hiciste en total para encontrar el 68? ¿Qué habría pasado si el arreglo no estuviera ordenado? ***Fueron 2 comparaciones y en caso de que no se encontrara mostraría un -1 dando a entender que el numero no existe.***

Etapa 2: Implementación en Java

Método 1: Búsqueda lineal

Completa el método para que recorra el arreglo y devuelva el índice del valor buscado, o -1 si no existe.

```
public static int busquedaLineal(int[] arr, int objetivo) {
```

```
// 1. Recorre el arreglo con un ciclo desde el índice 0 hasta arr.length - 1
// 2. Si arr[i] == objetivo, devuelve i
// 3. Si el ciclo termina sin encontrarlo, devuelve -1
}
```

Método 2: Búsqueda binaria

Completa el método usando los índices inicio, fin y medio. Puedes apoyarte en tu simulación de la Etapa 1.

```
public static int busquedaBinaria(int[] arr, int objetivo)
{      int inicio = 0; int fin = arr.length - 1;
    // 1. Mientras inicio <= fin:
    //    calcula medio = (inicio + fin) / 2
    //    si arr[medio] == objetivo, devuelve medio
    //    si arr[medio] < objetivo, mueve inicio a medio + 1
    //    si arr[medio] > objetivo, mueve fin a medio - 1
    // 2. Si el ciclo termina, devuelve -1
}
```

Traza de validación: Ejecuta mentalmente tu método busquedaBinaria con el arreglo del caso práctico y el objetivo 68. Anota el valor de inicio, fin y medio en cada vuelta del ciclo y verifica que coincide con tu simulación de la Etapa 1. Si no coincide, explica en qué línea está la diferencia.

Análisis de complejidad temporal (Big O)

Direcciones: Con base en el número de comparaciones que realizaste en el caso práctico, responde:

1. ¿Cuántas comparaciones hizo la búsqueda lineal en el peor caso para este arreglo de 8elementos? ¿Y la búsqueda binaria?

2. Si el arreglo tuviera 1 000 elementos, la lineal podría necesitar hasta 1 000 comparaciones,pero la binaria necesitaría alrededor de 10. Con esa idea, clasifica la complejidad de cada método (O(n) u O(log n)) y justifica por qué la binaria reduce el problema a la mitad en cada paso.

3. Un compañero quiere aplicar búsqueda binaria a un arreglo de IDs recién capturado, sinordenarlo. ¿Qué le explicarías sobre por qué esto no funciona, y qué debería hacer primero?

Nombre: Hernández Exiga Jesús Manuel **Fecha:** 13/11/26 **Clase:** GTID244

Síntesis de Conceptos y Autoevaluación

Este es el repaso final del paquete. Primero completarás una tabla que resume todo lo que estudiaste: las estructuras de datos lineales (lista, pila y cola) y los algoritmos de ordenamiento y búsqueda. Después responderás cinco situaciones del mundo real donde deberás elegir la herramienta más eficiente y justificar tu elección. Cierra con una reflexión escrita.

Parte 1: Tabla de Repaso

Instrucciones: Completa cada celda con tus propias palabras. Piensa en las operaciones que practicaste (push/pop, enqueue/dequeue, add/remove), en los principios LIFO y FIFO, y en cómo se comportó cada algoritmo en las etapas que resolviste.

Concepto	Características clave	Ventajas	Desventajas
Lista Enlazada	Add y Remove	Se desplaza en ambas direcciones y es eficiente	Ocupa más memoria para su uso
Pila (Stack)	estructura LIFO (Last In, First Out): el último elemento en entrar es el primero en salir.	Se puede dar a una banda de prioridad.	Sigue el orden sin importar si hay dato más importantes o necesarios
Cola (Queue)	estructura FIFO (First In, First Out): el primer elemento en entrar es el primero en salir.	Uso a informaciones de acciones rapidas, como van entrando sales y no importa su valor o orden.	Al ser datos o información que entra y sale como se ingresa, la información es inconsiza o sale revuelta.
Bubble Sort	recorre el arreglo comparando elementos vecinos y los intercambia si están en el orden incorrecto. En cada pasada, el valor más	Perfecta para un ordenamiento preciso y paciente	Es de orden lento no es tan rapido.

	grande "flota" hasta su posición final.		
QuickSort	elige un elemento llamado **pivote**, divide el arreglo en dos partes y repite el proceso en cada parte.	Es más eficiente porque hace menos comparaciones.	Ninguna, es ecxelente en su trabajo 10/10
Búsqueda Lineal	recorre el arreglo desde el primer elemento hasta el último, comparando uno por uno.	Es equitativo, no le importa si el arreglo esta ordenado o no este cumple con su recorrido sin excepción.	Es un recorrido que va de 1 en 1, es lento.
Búsqueda Binaria	compara el elemento buscado con el valor del medio y descarta la mitad del arreglo en cada paso.	Al estar ordenado puede decidir y buscar que mitad le srive que cual no, así descartando y buscando en la parte que necesita.	Esta condicionada a que la lista o datos tienen que estar forsozamente ordenados.

Parte 2: Situaciones del Mundo Real

Instrucciones: Lee cada situación y elige la opción más eficiente. Piensa en el principio que rige cada estructura y en la complejidad temporal de cada algoritmo.

1. Una app de mensajería debe mostrar los mensajes en el mismo orden en que llegaron: el primero que entra es el primero que sale. ¿Qué estructura es la más adecuada?

A) Una lista enlazada, porque permite eliminar elementos en cualquier posición.

B) Una cola, porque sigue el principio FIFO: el primer mensaje en llegar es el primero en salir.

C) Una pila, porque el último mensaje en llegar se procesa primero.

2. El botón 'Atrás' de un navegador web debe regresar a la página visitada más recientemente. ¿Qué estructura representa mejor ese historial?

A) Una lista enlazada, porque el historial puede crecer sin límite.

B) Una pila, porque la última página visitada queda arriba y es la primera en salir con pop.

C) Una cola, porque las páginas se visitan en orden cronológico.

3. Un sistema tiene un arreglo de 1,000,000 de IDs de usuario ya ordenados de menor a mayor y debe buscar uno de ellos con la menor cantidad de comparaciones posible. ¿Qué método conviene usar?

A) Búsqueda binaria, porque descarta la mitad del arreglo en cada comparación.

B) Búsqueda lineal, porque funciona sobre cualquier arreglo sin importar su orden.

C) Bubble Sort seguido de búsqueda lineal, porque ordenar siempre acelera la búsqueda.

4. Un programa debe ordenar un arreglo de 500,000 números lo más rápido posible. El orden previo de los datos es desconocido. ¿Qué algoritmo es la mejor elección de las dos vistas en clase?

A) Ambos tardan lo mismo porque los dos ordenan el arreglo completo.

B) Bubble Sort, porque compara elementos vecinos y es fácil de programar.

C) QuickSort, porque en promedio divide el problema en partes más pequeñas y es mucho máseficiente en arreglos grandes.

5. Una app debe revertir una cadena de texto, por ejemplo convertir 'hola' en 'aloh'. ¿Qué estrategia usa la estructura vista en clase?

A) Recorrer la cadena con búsqueda binaria para encontrar la última letra.

B) Apilar cada letra con push y luego extraerlas con pop, obteniendo las letras en orden inverso.

C) Insertar cada letra en una cola y extraerlas con dequeue.

Parte 3: Reflexión Final

Instrucciones: Responde con tus propias palabras. Usa lo que observaste en la simulación manual de la búsqueda binaria para apoyar tu explicación.

1. Explica la relación entre el ordenamiento y la eficiencia de la búsqueda: ¿por qué la búsqueda binaria solo es posible sobre datos ordenados, y qué pasaría con las comparaciones si el arreglo estuviera desordenado? Menciona al menos una situación del mundo real donde valga la pena invertir tiempo en ordenar antes de buscar.

Al tener un ordenamiento completo se tiene una mejor facilidad al momento de seleccionar y descartar información que no es necesaria y se evita el tener que estar buscando donde no es necesario buscar.

Si se usa el binario forsozamente se pide usar cuando esta ordenado, de lo contrario no funciona y da error.

Si es un caso donde valga la pena el sesgar la información, opinaria que seria si es necesario buscar a una persona en especial o algo necesariamente grave que se ocupe el ordenar y descartar información que no es necesaria.